

pmMDA

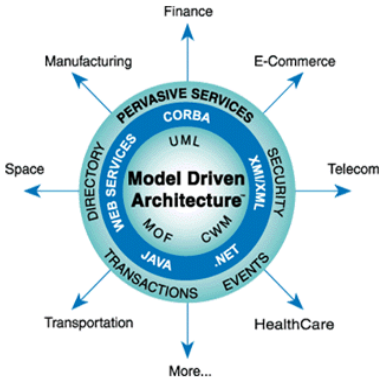
pmMDA
poor man MDA

Adrian Bachofen, bbv Software Services
Marcel Baumann, bbv Software Services

Contents

- MDD / MDA Overview
  - Advantages of a MDA approach
  - Limitations and drawbacks
- *pmMDA* an Open Source Implementation
  - Achieved Results
  - Open Source Approach
  - Limitations and next steps
- Open Discussion & Demonstration

MDD/MDA Principles



How Systems Will Be Built

MDA® provides an open, vendor-neutral approach to the challenge of business and technology change. Based firmly upon OMG's established standards*, MDA aims to separate business or application logic from underlying platform technology. Platform-independent applications built using MDA and associated standards can be realized on a range of open and proprietary platforms, including CORBA®, J2EE, .NET, and Web Services or other Web-based platforms. Fully-specified platform-independent models (including behavior) can enable intellectual property to move away from technology-specific code, helping to insulate business applications from technology evolution, and further enable interoperability. In addition, business applications, freed from technology specifics, will be more able to evolve at the different pace of business evolution.

MDD / MDA Principles

- PIM *P*latform *I*ndependent *M*odel
- PSM *P*latform *S*pecific *M*odel
- UML *U*nified *M*odeling *L*anguage
- XMI *X*ML *M*etadata *I*nterchange
- OMG *O*bject *M*anagement *G*roup
- MOF *M*eta *O*bject *F*acility

bbv behind things.

MDA – Code Generation

- The designers work with the business model. The model is not polluted with platform aspect.
- Model changes are fast and cost effective - PIM to source code artifacts -
- Designers love code generators. Programmers don't like them
- *pmMDA implements a subset of MDA approach. No PSM is supported.*

bbv behind things.

Code Generators

Advantages

- Design model and source code are synchronized
- Streamline and unify coding guidelines
- Eliminate copy & paste errors
- No side effects in getters and setters
- Allow controlled extensions

Drawbacks

- Maintainers are responsible for the generators
- Does not support own coding styles and approaches
- Complex logic must be implemented separately

MDA in an Insurance Project

- Business model changed every two weeks during eighteen months – about 36 changes impacting user interface, client server communication, database schema, business model
- Business roles changed every two weeks and were not fully specified – about 1200 rules were regularly updated
- The application is deployed in fifteen environments – separate server, database, version

MDA in an Insurance Project

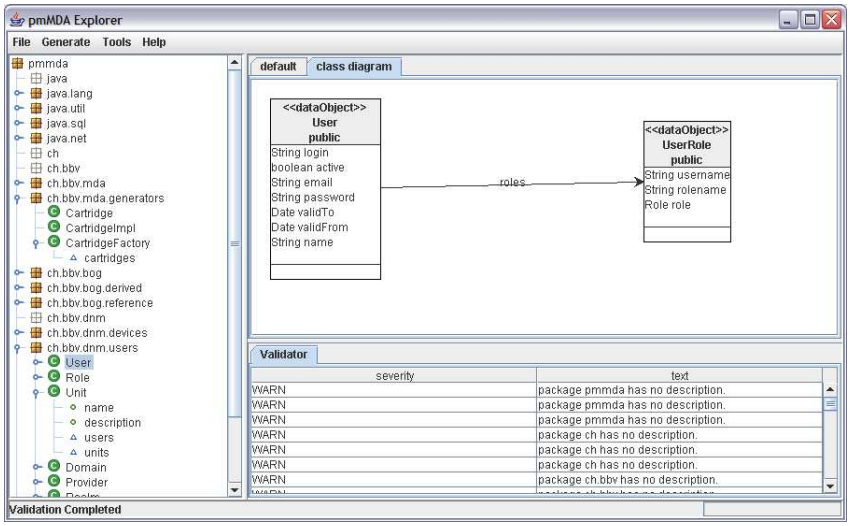
Gains

- Business model, database, communication layers are automatically generated
- Documentation is automatically updated
- Only one document to modify to implement one change
- Incompatible changes are detected at compile time

Losses

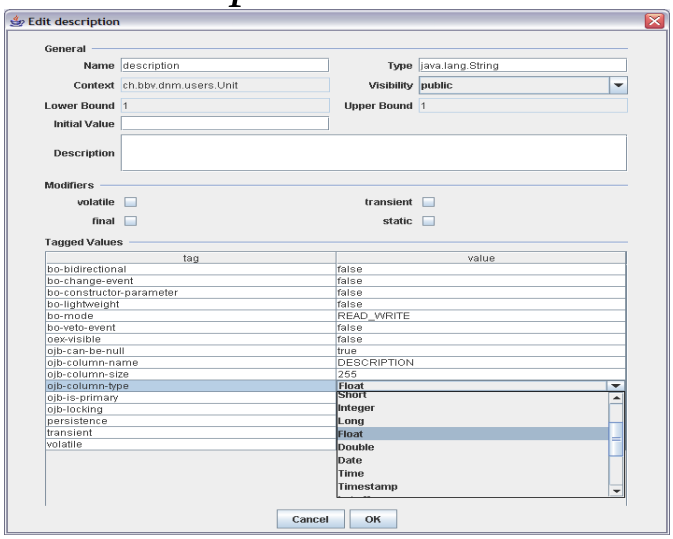
- Programmers never fully understood the concept and therefore fought against
- Changes in the code generators had complex consequences – visitor pattern semantic change -
- Database DB2 do not like drop of complete database – extensive and slow operation -

pmMDA

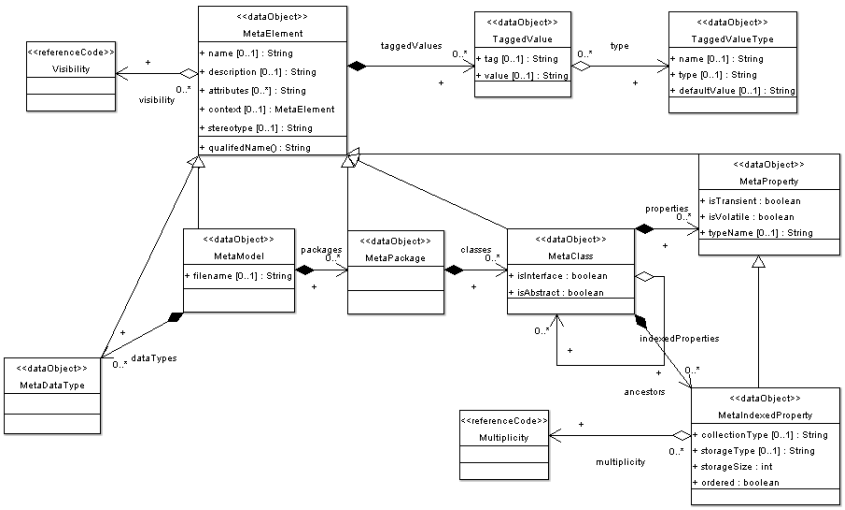


bbv behind things.

pmMDA


Edit description
General
Name
description
Type
java.lang.String
Context
ch.bbv.dnm.users.Unit
Visibility
public
Lower Bound
1
Upper Bound
1
Initial Value
Description
Modifiers
volatile
transient
final
static
Tagged Values
tag
value
bo-bidirectional
false
bo-change-event
false
bo-constructor-parameter
false
bo-lightweight
false
bo-mode
READ_WRITE
bo-veto-event
false
oex-visible
false
ojb-can-be-null
true
ojb-column-name
DESCRIPTION
ojb-column-size
255
ojb-column-type
Float
ojb-is-primary
ojb-locking
persistence
transient
volatile
Short
Integer
Long
Float
Double
Date
Time
Timestamp
Cancel
OK

pmMDA



bbv behind things.

pmMDA

- Goals
  - UML compatible
- Ideas
  - Persistence layer
  - Business model and SOA based
  - UML model oriented
- Targets
  - 2 tiers, CORBA, J2EE/RMI, Swing explorer interface

pmMDA Frameworks

- Data Object Graph
  - Data Transfer Object, Graph DTO, Lightweight
- Explorer Framework
  - Declarative model, Meta Types, Reflection
- Application
  - Locator, startup & shutdown
  - Validation
  - Command line

bbv behind things.

Open Source Projects

- Development Tools
  - JDK, Eclipse, Ant, CVS, JavaDoc, Forrest, etc.
  - Source forge
- Design Tools
  - ArgoUML, Poseidon
  - Open Office
- Quality Assurance
  - Metrics, JUnit, Logger, DocCheck, etc.

Open Source Libraries

- Java Libraries
  - Apache Jakarta – Tomcat, Velocity, bean utilities, logger, etc.
  - Others: graph, forms, glazed lists, etc
- Databases
  - MySQL, PostgresSQL, Derby
- Interfaces
  - NSUML, MDR, JMI, CORBA, XSTREAM, Xerces

Open Source Philosophy

- Meritocracy
  - Show what you can. If it is good you receive the rights to do more
- One person owns a package
  - He can delegate ownerships. Nobody else has the right to modify models or source code
- Artifacts checked in compiles and run

Open Source Philosophy

- Everything is accessible
  - For example using CVS
  - You are not happy with the project, make a copy and make your own changes or start from scratch
- Open source developers often do not like Microsoft
  - Don't use closed source or standards
  - Secrets are bad – example NTFS is not documented -
  - Some of these developers behave childish

bbv behind things.

Open Discussion

- MDA/MDD
- pmMDA
- Open Source
- Demonstration
  - *pmMDA*
  - *Example*
  - *pmmda.sourceforge.net*